ETH zürich

Policy Gradient & REINFORCE

Suei-Wen Chen

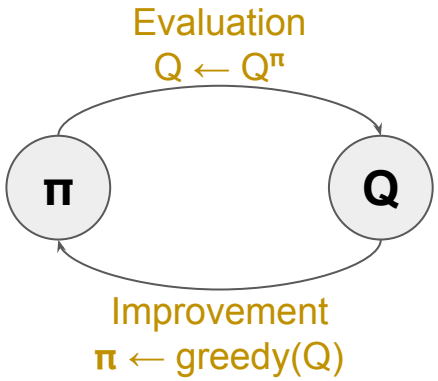
Weekend 4, From Data to Solutions

Summer 2026

Roadmap of This Weekend

1. Introduction
2. Policy Gradient Methods: REINFORCE & Baselines
3. Variance Reduction: Actor-Critic
4. Credit Assignment: GAE and PPO
5. Human Alignment & Reasoning: RL in LLM
6. New Paradigm: Safe/Risk-Aware RL
7. Project Release

Recall: Value Functions and Policies



Goal of RL: Learn an optimal policy π^* via trial and error

Before: We used TD-Learning to compute the Q function

… whose greedy policy leads to improvement

But this is an ***indirect*** way to obtain the policy (derived from Q)

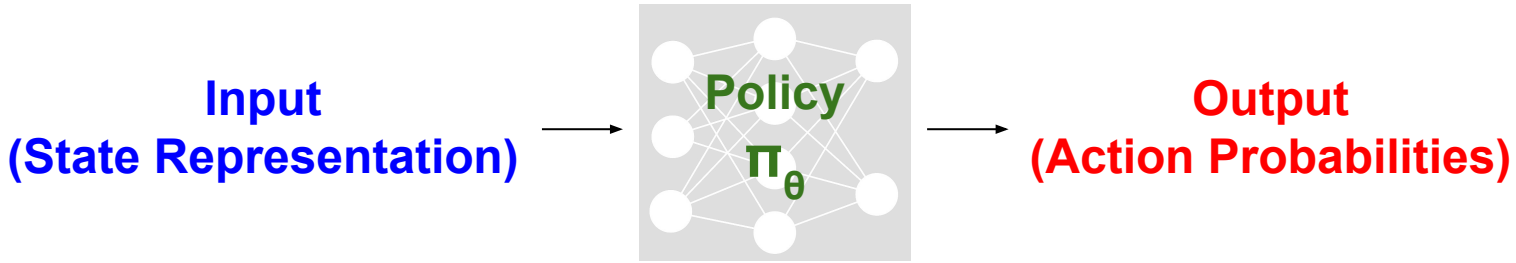
Issue: Computing greedy policy is too expensive for large action spaces

Alternative approach: *Directly* learn a policy without using the value function

	Value-Based Approach	**Policy-Based Approach**
Focus	Estimate value functions to derive policies	Maximize probabilities of good actions

Policy-Based Approach in RL

Idea: Train a parametrized policy π_θ without estimating the value function



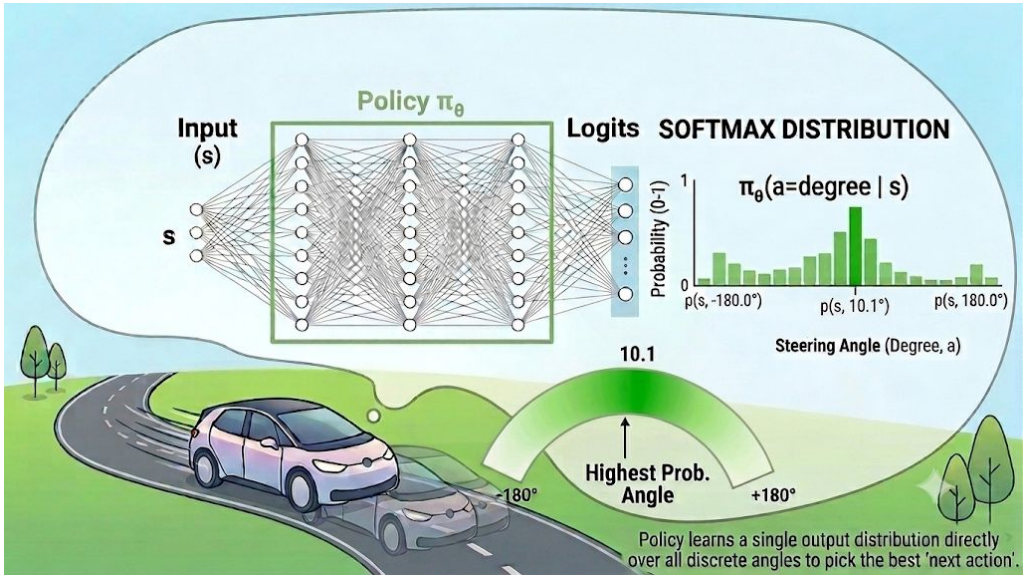
Examples	State	Action
Autonomous car	images & sensor readings	acceleration & steering angle
Chess	current board position	next move

Example: Control the steering angle of a car

All possible actions: [-180°,+180°]

Idea: Discretize the space

$$
\begin{aligned}
a_1 &= -180.0° \\
a_2 &= -179.9° \\
&\vdots \\
a_{3600} &= +179.9° \\
a_{3601} &= +180.0°
\end{aligned}
$$



Policy: NN with a *softmax* final layer $\mathrm{SoftMax}(x_1, \ldots, x_M) = \left(\frac{e^{x_1}}{e^{x_1} + \cdots + e^{x_M}}, \ldots, \frac{e^{x_M}}{e^{x_1} + \cdots + e^{x_M}} \right)$

Discretization is flexible but expensive and loses sense of “distance”

Example: Control the steering angle of a car (conti.)

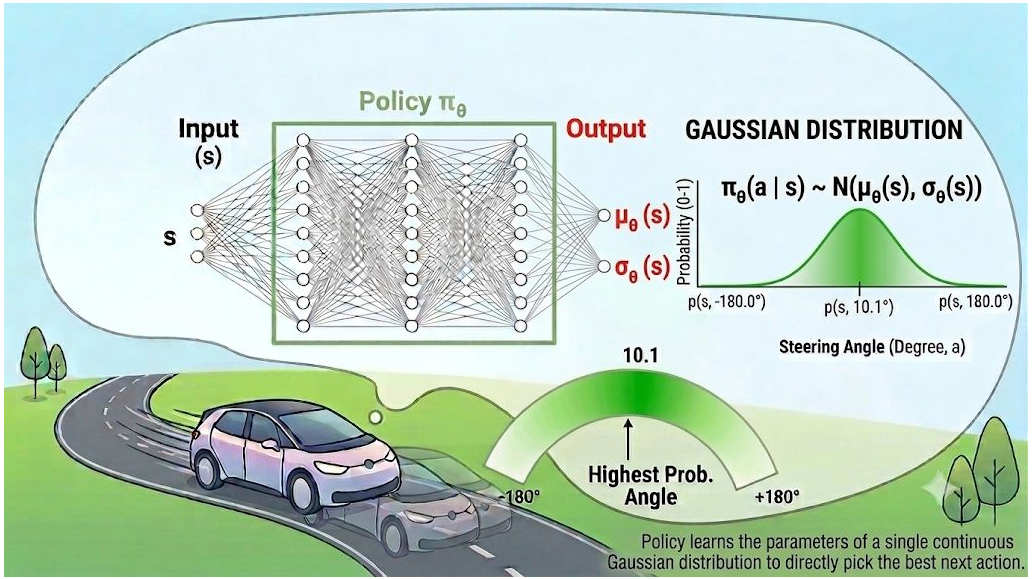
All possible actions: [-180°,+180°]

Idea: assume actions follow a (clipped) normal distribution

NN: outputs the mean $\mu_\theta(s)$ and std $\sigma_\theta(s)$ of the distribution

Policy: choose action from

$\pi_\theta(a|s) = N(a; \mu_\theta(s), \sigma_\theta^2(s))$



How to train a parametrized policy π_θ

Goal: Find parameters **θ** which defines a policy π_θ with the largest value function

Performance measure J(θ): One number measuring the quality of **θ** across initial states s_0 (with distribution **ρ**)

Average over initial state s_0

quality of π_θ at state s_0

$G(\tau) = \sum_{t=1}^{T} \gamma^{t-1} R_t$: return of a sample trajectory **τ**

$$J(\theta) := \mathbb{E}_{S_0 \sim \rho}[V^{\pi_\theta}(S_0)] = \mathbb{E}^{\pi_\theta}_{S_0 \sim \rho}[G(\tau)]$$

How: Perform gradient ascent on **J(θ)**

Q: How to compute this?

Next Aim — Learning Rate

$$\theta \leftarrow \theta + \alpha_t \nabla_\theta J(\theta)$$

Current Aim — Direction towards the target

Policy Gradient Theorem

In one sentence: The improvement direction can be written as a combination of the gradients of the (log) policy weighted by return.

If $\tau : S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T, S_T$ is a trajectory following $\boldsymbol{\pi_\theta}$:

weighted average based on how likely a trajectory is

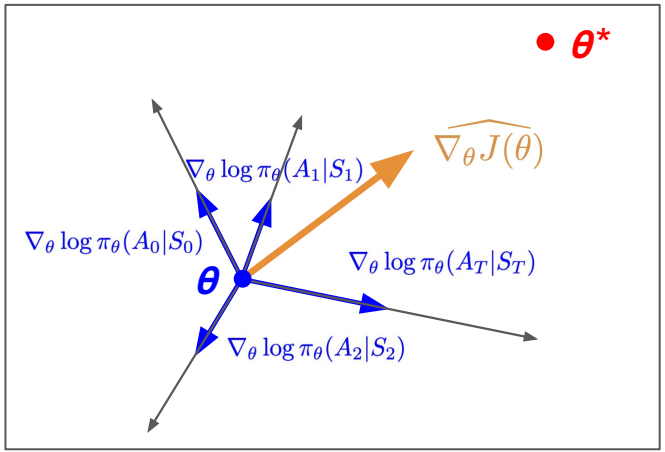
score function: direction increasing the probability of taking action A_t at state S_t

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} G(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]$$

improvement direction

combination of directions weighted by return

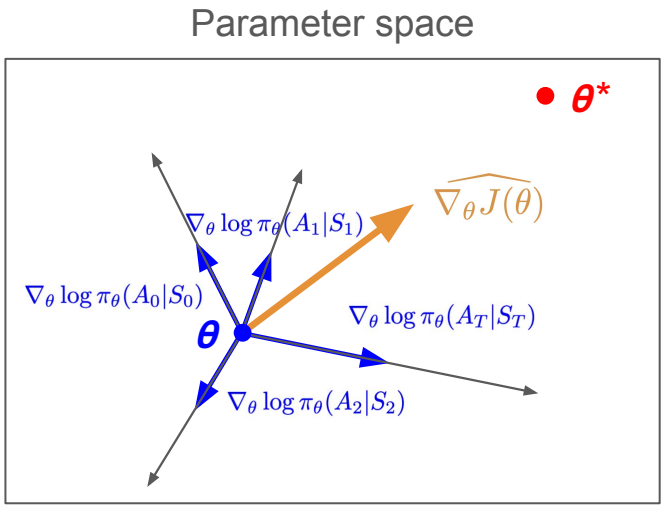
Parameter space



Issue: Violation of Causality

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} G(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]$$

Update directions of later actions are contaminated by past rewards



Example (Chess):

Correctly punished

Incorrectly punished

Move 0	..	Move 20	Move 21	Move 22	..	Move 30	Loss
Blunder	.	Blunder	Blunder	Best	.	Best	G(T)=-1

Punish all past actions indiscriminately (same scaling for all score functions)

Fix: Causality Trick

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} G(\tau) \nabla_\theta \log \pi_\theta(A_t | S_t) \right]$$

Idea: At each step t, replace the scaling $G(\tau)$ by the ***reward-to-go*** $\gamma^t G_t(\tau)$

$$G(\tau) = \underbrace{R_1 + \gamma R_2 + \cdots + \gamma^{t-1} R_t}_{\text{past rewards } b(\tau_{0:t-1})} + \underbrace{\gamma^t \left(R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T \right)}_{\text{future rewards } \gamma^t G_t(\tau)}$$

Here, $b(\tau_{0:t-1})$ is called a ***baseline*** which helps us isolate downstream rewards

$$G(\tau) - b(\tau_{0:t-1}) = \gamma^t G_t(\tau)$$

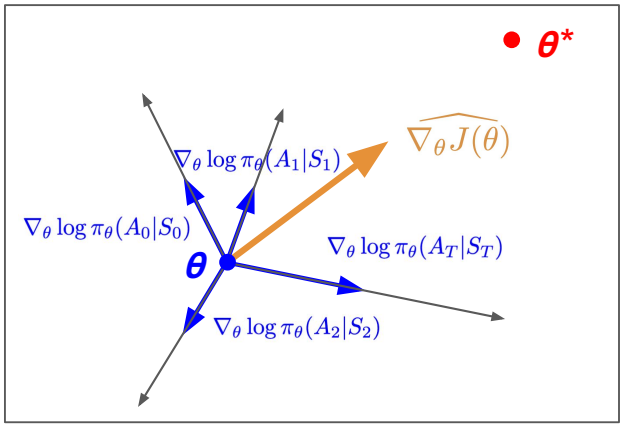
Fix: Causality Trick (cont'd)

The improvement direction can be written as:

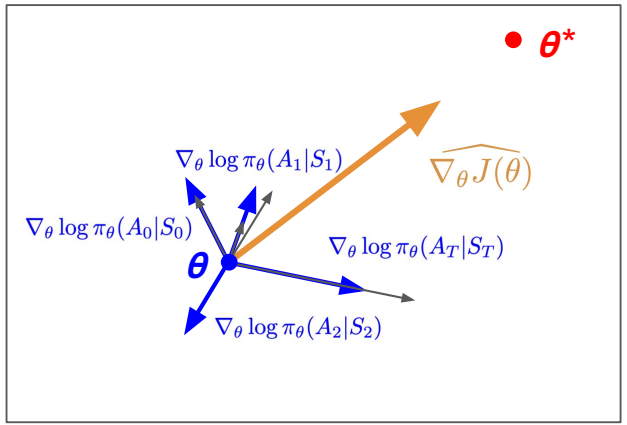
Different scaling for each step `t`
(only depending on future)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}(G(\tau) - b(\tau_{0:t-1}))\nabla_\theta \log \pi_\theta(A_t|S_t)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t G_t(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)\right]$$

Original (without baseline)



With Baseline $b(\tau_{0:t-1})$



For a random trajectory:

- Both are correct on average **(bias=0)**
- Baselined version tends to be more accurate! **(lower variance)**

The REINFORCE Algorithm

Idea: Approximate the improvement direction by sampling, using the formula

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t G_t(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]$$

Algorithm: At iteration k, sample a trajectory **τ** from policy **π_θ** and perform gradient ascent as follows:

Next Aim — Learning Rate — Improvement Direction (gradient estimate)

$$\theta \leftarrow \theta + \alpha_k \left[\sum_{t=0}^{T-1} \gamma^t G_t(\tau) \nabla_\theta \log \pi_\theta(A_t|S_t) \right]$$

Current Aim

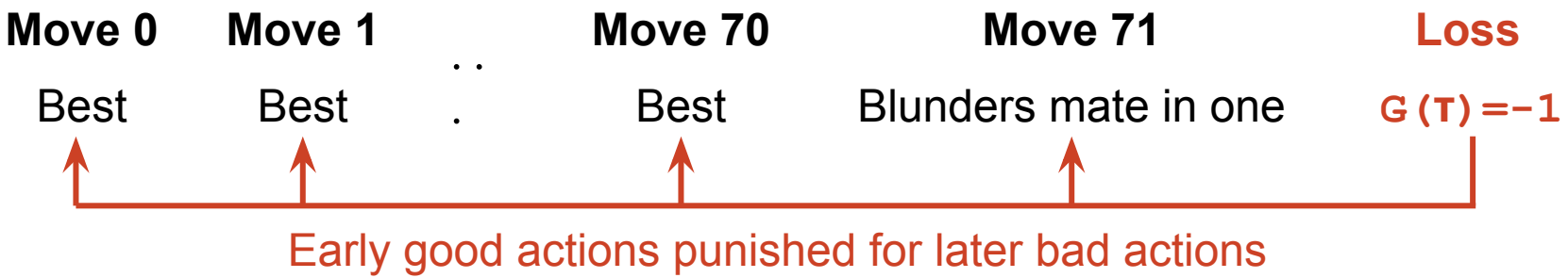
Issue: Lack of Credit Assignment

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t G_t(\tau)\nabla_\theta \log \pi_\theta(A_t|S_t)\right]$$

Each action is weighted by ***all*** downstream rewards

→ unreliable/unstable indicator of performance (noise, luck, …)

Example (Chess):



Fix: Add another baseline to isolate the contribution of each action!

This gives rise to the ***REINFORCE with Baseline*** algorithm.

Fix: State-dependent Baseline

Find a baseline $b(S_t)$ to isolate the contribution of $\mathbf{A_t}$

	REINFORCE	**REINFORCE with baseline**
Question Asked:	*Did this action $\mathbf{A_t}$ lead to a high positive return?*	*Was this action $\mathbf{A_t}$ better or worse than what we normally expect from state $\mathbf{S_t}$?*
Performance Indicator for $\mathbf{A_t}$:	reward-to-go $\gamma^t G_t(\tau)$	"advantage" $\gamma^t (G_t(\tau) - b(S_t))$

Choices for $\mathbf{b(S_t)}$:

1. Sample mean of returns from multiple rollouts (starting at $\mathbf{S_t}$)
2. Another NN $\mathbf{v_\varphi}$ approximating $\mathbf{v^\pi}$

Summary: Baselines

Baselines give us better gradient estimates for policy improvement by

1. Reducing the variance of the gradient estimator
2. Achieving better credit assignment for individual actions

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t (G_t(\tau) - b) \nabla_\theta \log \pi_\theta(A_t | S_t) \right]$$

Possible Choices	Example	Intuition
Constant value b	Sample mean of multiple returns	Centering reduces variance
Function of the past $b(\tau_{0:t-1})$	Cumulative rewards until now	Isolate future rewards
State-dependent $b(s_t)$	Estimate of $v^\pi(s_t)$	Advantage of an action

Summary: Policy-Gradient Methods

All policy gradient methods estimate the following expectation:

Performance indicator of an action $A_t \sim \pi_\theta(\cdot|S_t)$

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \Psi_t \nabla_\theta \log \pi_\theta(A_t|S_t)\right]$$

Algorithm	Ψ_t	Bias	Variance
REINFORCE	Reward-to-go $\gamma^t G_t$	Unbiased	High
REINFORCE with Baseline	Reward-to-go with baseline $\gamma^t(G_t - b(S_t))$	Unbiased	Moderate
Actor-Critic	One-step TD error	Biased	Low
Generalized Advantage Estimate	Mixture of n-step TD errors	Tunable	Tunable